

使用 Antigravity 和 Spec-kit 依規格開發 ADK 代理

來源：<https://codelabs.developers.google.com/sdd-adk-antigravity?hl=zh-tw>

步驟數：8

透過 Antigravity 的 Spec 導向開發工作流程，擴充現有的餐廳服務 ADK 代理程式，加入預訂功能。您將瞭解如何將領域知識封裝為 Antigravity 技能、設定專案章程、啟動專案環境以進行 AI 輔助開發，以及使用「指定-釐清-規劃-工作-分析-實作」週期，有系統地將功能新增至現有程式碼集。

目錄

1. [1. 簡介](#)
2. [2. 設定環境](#)
3. [3. 探索範例程式碼並瞭解規格導向開發](#)
4. [4. 使用 Antigravity 啟動專案環境](#)
5. [5. SDD 週期 - 新增預留功能](#)
6. [6. 實作](#)
7. [7. 挑戰 \(選填\)](#)
8. [8. 恭喜 !](#)

1. 簡介

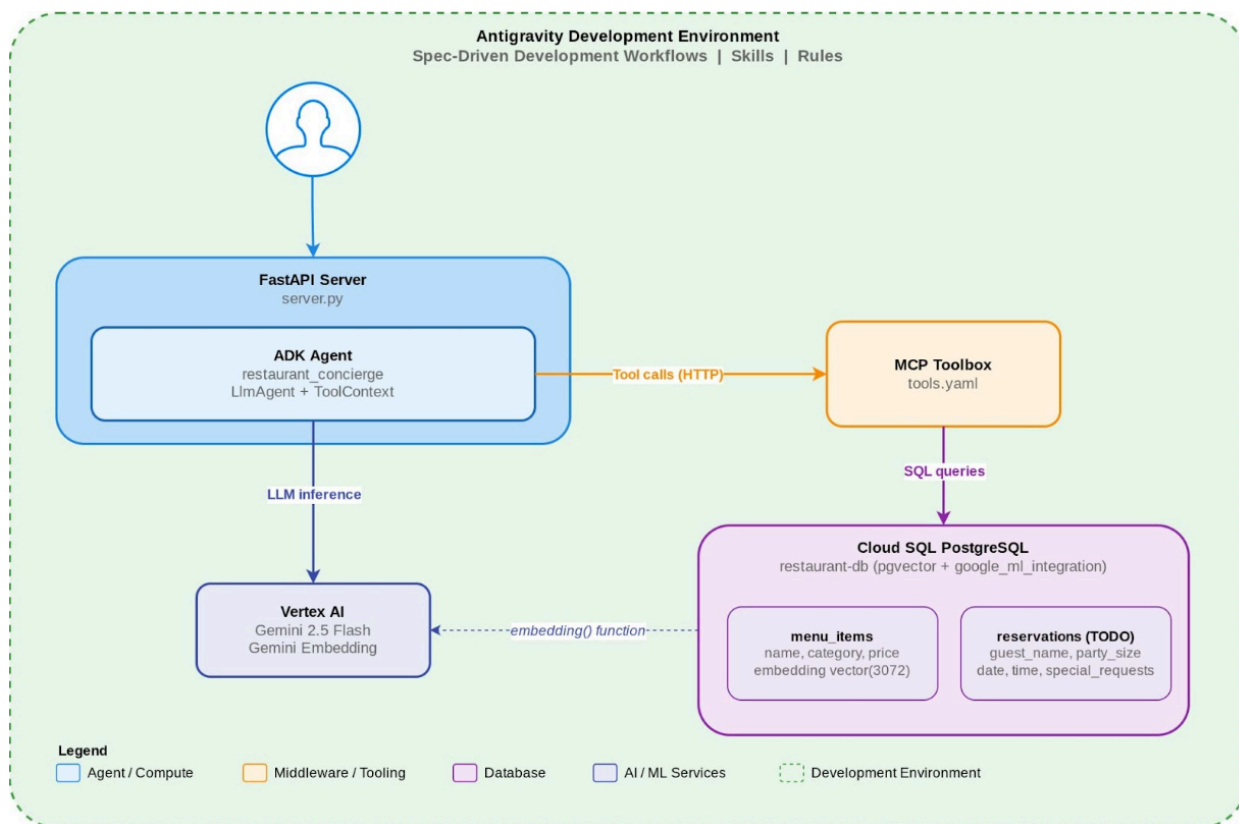
為現有代理程式新增功能 (以資料庫為基礎的新功能) 通常需要編寫樣板、整合整合項目，並確保所有內容與程式碼集中的模式一致。Antigravity 可加速這項程序的每個階段：分析程式碼集以建立所需脈絡、產生結構化規格和實作計畫供您審查，以及執行程式碼變更。所有作業都會以領域知識為指引，協助您擷取可重複使用的技能，並制定專案章程來強制執行不可協商的原則。本程式碼實驗室介紹如何透過導入新週期，大幅參考 [spec-kit](#)，加強規格文件，進而提升 Antigravity 規格導向開發範例的效能。

建構項目

在本地執行的餐廳服務應用程式，透過完整的 SDD 週期新增預訂功能：

- **預訂訂位**：顧客可預訂座位及查看預訂資訊，這項功能由新的 MCP Toolbox 資料庫工具和 Cloud SQL `reservations` 資料表提供支援
- **(挑戰)** – 為代理程式開發專屬 UI
- **(挑戰)** – 在 Antigravity 代理程式的協助下部署至 Google Cloud

範例程式碼提供可運作的 ADK 代理程式，其中包含菜單搜尋 (透過 MCP Toolbox 進行關鍵字 + 語意搜尋) 和飲食偏好追蹤 (透過 ToolContext)。您無需手動編寫應用程式程式碼即可擴充，Antigravity 會根據您的規格處理實作作業。



課程內容

- 如何啟動專案環境，讓 Antigravity 瞭解現有程式碼
- 如何建立 Antigravity 技能，以及設定 MCP，將網域知識 (例如 ADK 文件、第三方技能) 打包以供重複使用
- 如何設定專案章程，供 SDD 工作流程在規劃和分析期間進行驗證
- 瞭解如何在 Antigravity 中使用 Spec-Driven Development (SDD) 工作流程，有系統地新增功能
- 如何透過 MCP Toolbox，使用新的資料庫支援工具擴充 ADK 代理

必要條件

- 本機電腦已安裝 [Google Antigravity](#) 和 `git`
 - 已啟用有效帳單帳戶的 Google Cloud 帳戶
 - 建議您先完成四個先決條件 ADK 程式碼研究室 (或具備同等知識)，有助於瞭解用途情境：
 - [使用 ADK 建構 AI 代理：基礎知識](#)
 - [使用 ADK 建構 AI 代理：加入工具](#)
 - [使用 ADK 和 CloudSQL 建構持續性 AI 代理](#)
 - [在 Cloud Run 部署、管理及觀測 ADK 代理](#)
 - [資料庫即工具：運用 ADK、MCP Toolbox 和 Cloud SQL 建構代理式 RAG](#)
-

步驟 2 · 約 12 分鐘

2. 設定環境

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

重要注意事項：

本教學課程需要具備有效帳單帳戶的 Google Cloud 專案。如果沒有，請使用本程式碼研究室頂端的橫幅，取得可用於本教學課程的試用帳單帳戶。

這個步驟會複製入門存放區、向 Google Cloud 進行驗證、佈建 Cloud SQL 資料庫，並準備本機 Antigravity 環境。

注意：本程式碼研究室完全在本機電腦的 Google Antigravity 中執行。設定程序包括使用 `gcloud` CLI 指令手動驗證

安裝必備條件

請確認系統具有下列 CLI 可執行檔：

- `git`
- `gcloud`
- `uv`
- `npm`

安裝說明會因電腦的作業系統而異。

複製範例存放區

在 Antigravity (或系統終端機) 中開啟終端機。複製隨附存放區並輸入目錄：

```
git clone https://github.com/alphinside/sdd-adk-antigravity-starter.git sdd-adk-agents-agy
cd sdd-adk-agents-agy
```

在 Antigravity 中開啟複製的存放區。依序點選「File」->「Open Folder」->選取複製的目錄「sdd-adk-agents-agy」

移除上游遙控器。SDD 工作流程會為功能規格建立 git 分支版本，移除遠端可避免意外推送至起始存放區：

```
git remote remove origin
```

重要事項：本程式碼研究室中的 SDD 工作流程會使用 `git` 分支機構來整理功能規格。移除 `origin` 遠端可確保本機分支版本和修訂版本保留在本機。如要備份工作，請新增自己的遠端：`git remote add origin <your-repo-url>`

使用 Google Cloud 進行驗證

執行兩項驗證指令。兩者都會開啟瀏覽器進行 OAuth：

```
gcloud auth login
gcloud auth application-default login
```

注意：在第二個 OAuth 指令中，系統會提示您選取權限。查看所有選項

注意：請務必使用申請試用帳單帳戶的電子郵件地址進行驗證

由於您是在本機使用 Antigravity，因此請手動驗證。`auth login` 會驗證 `gcloud CLI`。`application-default login` 驗證應用程式使用的 Google Cloud SDK，包括 ADK 的 Vertex AI 呼叫和 Cloud SQL Python 連接器，兩者都依賴應用程式預設憑證。

設定 Google Cloud 專案

在執行專案設定指令碼前，請先將位置變數寫入 `.env`：

```
echo "GOOGLE_CLOUD_LOCATION=global" > .env
echo "REGION=us-central1" >> .env
```

- `GOOGLE_CLOUD_LOCATION=global` 用於 Vertex AI / Gemini API 呼叫。
- `REGION=us-central1` 用於 Cloud SQL 和其他 GCP 基礎架構

重要事項！以下步驟旨在協助您快速設定連結至試用帳單帳戶 (嚴格) 的 Google Cloud 專案。如要使用先前的專案，可以略過下列步驟，改為執行下列步驟：

在 `.env` 檔案中，將自己的專案名稱新增為 `GOOGLE_CLOUD_PROJECT` 變數

使用 `gcloud config set project your-project-id` 在終端機中啟用專案

然後直接跳到「啟用 API」

如果不確定，請繼續閱讀下節

下載並執行專案設定指令碼。這會建立或驗證具有試用帳單的 Google Cloud 雲端專案，並將專案 ID 儲存至 `.env`，然後提供來源：

```
curl -sL https://raw.githubusercontent.com/alphinside/cloud-trial-project-  
setup/main/setup_verify_trial_project.sh -o setup_verify_trial_project.sh  
  
bash setup_verify_trial_project.sh && source .env
```

正在啟用 API

啟用必要的 API：

```
gcloud services enable \  
  aiplatform.googleapis.com \  
  sqladmin.googleapis.com \  
  compute.googleapis.com \  
  cloudresourcemanager.googleapis.com
```

注意：啟用 API 可能需要一些時間。這項操作每個專案只需執行一次。

安裝依附元件

我們將使用 `uv` 做為 Python 專案管理員。`uv` 是以 Rust 編寫的快速 Python 套件和專案管理員 ([文件](#))。本程式碼研究室使用 `uv` 是為了簡化程序並加快速度。安裝 Python 依附元件：

```
uv sync
```

然後使用專案設定更新 ADK 代理程式的 `.env` 檔案：

```
cat > restaurant_concierge/.env <<EOF  
GOOGLE_CLOUD_PROJECT=${GOOGLE_CLOUD_PROJECT}  
GOOGLE_CLOUD_LOCATION=global  
GOOGLE_GENAI_USE_VERTEXAI=True  
EOF
```

準備資料庫基礎架構和資料

設定資料庫密碼並新增至 `.env`：

```
export DB_PASSWORD=codelabpassword  
echo "DB_PASSWORD=${DB_PASSWORD}" >> .env
```

重要事項：上述硬式編碼密碼僅供教學用途。在正式環境中，請使用 [Secret Manager](#) 等服務安全地儲存憑證，並在執行階段參照這些憑證，切勿將純文字密鑰提交至原始碼控管。

接著，執行 `scripts/setup_database.sh` 指令碼，準備好所有必要基礎架構。這項操作會執行下列動作：

- 建立 Cloud SQL 執行個體
- 檢查執行個體是否準備就緒
- 授予 Vertex AI 權限
- 建立資料庫
- 填充資料庫
- 在背景執行 MCP Toolbox 服務

```
chmod +x scripts/setup_database.sh  
./scripts/setup_database.sh > database_setup.log 2>&1 &
```

這項指令會在背景執行設定，您可以定期檢查 `database_setup.log` 檔案的輸出內容

現在，我們應該擁有所有必要的入門 ADK 代理程式存放區，可以開始作業。現在，在等待一切就緒的同時，我們將在下一節中進一步討論 Antigravity 和規格導向開發

步驟 3 · 約 10 分鐘

3. 探索範例程式碼並瞭解規格導向開發

本步驟會逐步說明範例程式碼結構、介紹規格導向開發方法、植入資料庫，並驗證基本代理程式是否正常運作，再開始擴充。

專案架構

在 Antigravity 編輯器中開啟複製的存放區專案，然後檢查目錄版面配置：

```
sdd-adk-agents-agy/
├── .agents/
│   ├── workflows/                # SDD slash commands (/speckit.*) – manual
trigger
│   │   ├── speckit.specify.md
│   │   ├── speckit.clarify.md
│   │   ├── speckit.plan.md
│   │   ├── speckit.tasks.md
│   │   ├── speckit.analyze.md
│   │   ├── speckit.implement.md
│   │   ├── speckit.checklist.md
│   │   └── speckit.constitution.md
│   └── skills/                   # Antigravity skills (loaded on demand, agent
determined)
│       ├── mcp-toolbox-postgres/
│       │   └── SKILL.md          # MCP Toolbox config skill
│       └── repo-research/
│           └── SKILL.md          # Repo analysis skill
├── rules/                        # Always-active context
├── .specify/                     # spec-kit SDD templates and memory
│   ├── memory/constitution.md
│   ├── templates/
│   └── scripts/
├── restaurant_concierge/        # ADK agent package
│   ├── __init__.py
│   ├── agent.py                 # LlmAgent + ToolContext tools + Toolbox integration
│   └── .env                     # Vertex AI configuration
├── server.py                    # FastAPI server wrapping the agent
├── tools.yaml                   # MCP Toolbox tool definitions
├── scripts/                     # Setup scripts
└── pyproject.toml
```

金鑰檔案

代理應用程式檔案

- **restaurant_concierge/agent.py**：核心代理程式。LlmAgent，結合 MCP Toolbox 資料庫工具和以 ToolContext 為基礎的飲食偏好追蹤功能。代理會從 Toolbox 伺服器載入所有工具，並新增兩個使用 ToolContext 管理狀態的 Python 函式 (save_dietary_preference 、 get_dietary_preferences)。

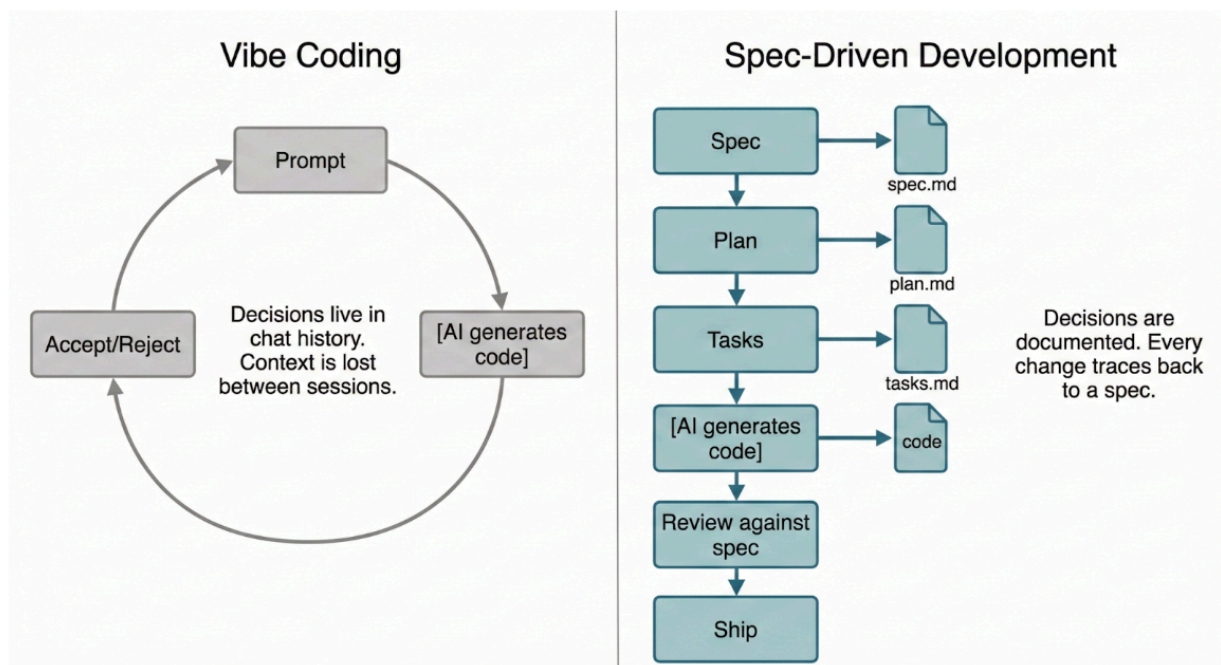
- **tools.yaml** - MCP Toolbox 工具定義。定義三種選單搜尋工具：關鍵字搜尋 (`search_menu`)、透過 `pgvector` 進行語意搜尋 (`semantic_search_menu`) 和類別篩選器 (`get_menu_by_category`)。目前沒有預訂工具，您稍後會新增這些工具
- **server.py**：最簡單的 FastAPI 伺服器，展示如何以 FastAPI 物件存取 ADK。ADK 的 `get_fast_api_app()` 提供內建端點，包括 SSE 串流和工作階段管理 API 的 `/run_sse`。

Antigravity 檔案

- **.agents/skills/mcp-toolbox-postgres/SKILL.md**：這項技能可引導 Antigravity 建立 MCP Toolbox 適用的 `tools.yaml` 設定。這項技能會說明建構適當 RAG 管道所需的 `sources`、`tools` 和 `embeddingModels` 設定。您必須新增適當的 YAML 前置事項，讓 Antigravity 辨識，這項技能才會啟用
- **.agents/skills/repo-research/SKILL.md**：這項技能會教導 Antigravity 如何逐步分析存放區，並產生結構化專案內容文件。這項工具採用 4 階段做法：表面掃描 (僅限目錄樹)、設定和中繼資料檔案、進入點和資料模型，然後是深入探討目標，每個階段都會停止並寫入發現項目，然後再繼續下一個階段。此外，您必須稍後新增 YAML 前言，才會啟用這項功能。啟用後，即可叫用這項功能來生成 `.agents/rules/project-context.md`，這份文件會全面介紹架構、執行階段依附元件、API 介面和網域詞彙，協助使用者順利完成設定程序。

規格導向開發：從 Antigravity 內建的規劃到結構化 SDD

AI 程式碼助理可根據提示輕鬆生成程式碼。風險：您用一句話描述功能，助理寫出數百行程式碼，您因為看起來沒問題而接受。這種做法有時稱為「直覺式程式開發」：您會根據感覺來引導，並根據輸出內容是否可行來接受或拒絕。可快速製作原型和拋棄式指令碼。當程式碼集變大、功能互動，或是您在幾週後重新查看程式碼，卻無法重現當初的決策原因時，就會發生這種情況。

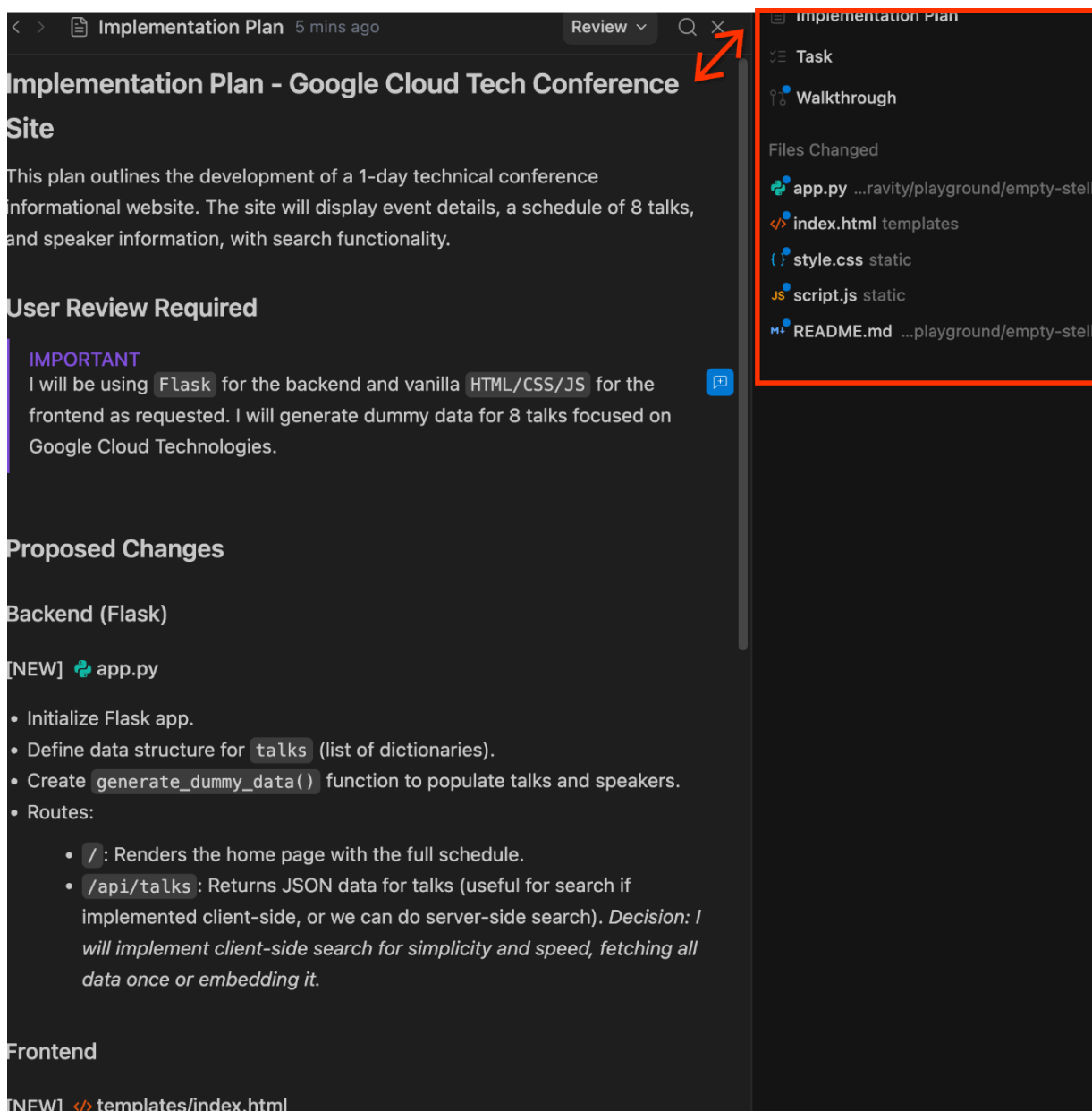


規格導向開發 (SDD) 會為這個迴圈新增結構。在生成任何程式碼前，請先撰寫規格：功能用途、服務對象、成功條件。AI 助理會根據規格運作，您在審查輸出內容時也是如此。規格會成為意圖的單一可靠資料來源。如果程式碼與規格不同，您可以在審查時發現。如果需求條件有異動，請先更新規格，然後重新產生。決策有據可查，而非臨時起意。

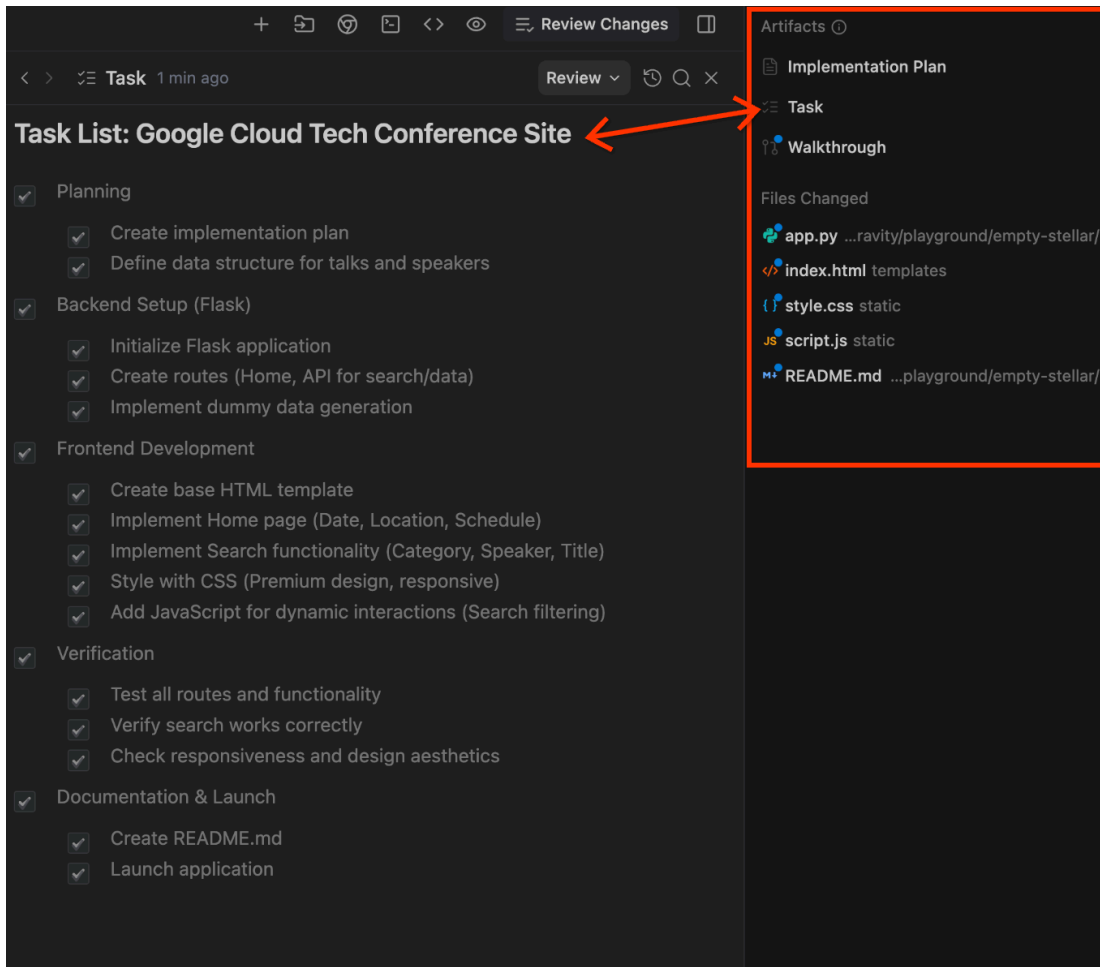
這項取舍是實際存在的：就每項功能而言，SDD 的速度比直覺式程式開發慢。您會先撰寫文件，再編寫程式碼。但這項投資會帶來複利效應：程式碼集每次變更都有脈絡可循，每次 AI 生成的實作內容都有可供審查的合約，而且只要向協作者 (人類或 AI) 提供規格，即可讓他們加入專案，不必從記憶中回想並說明決策。

Antigravity 已經遵循規格導向開發原則。將代理程式設為**規劃模式**後，系統會先產生兩項構件，再編寫任何程式碼：

- **導入計畫**：建議的技術方法、檔案變更和架構決策總覽



- **工作清單**：工作項目的結構化細目

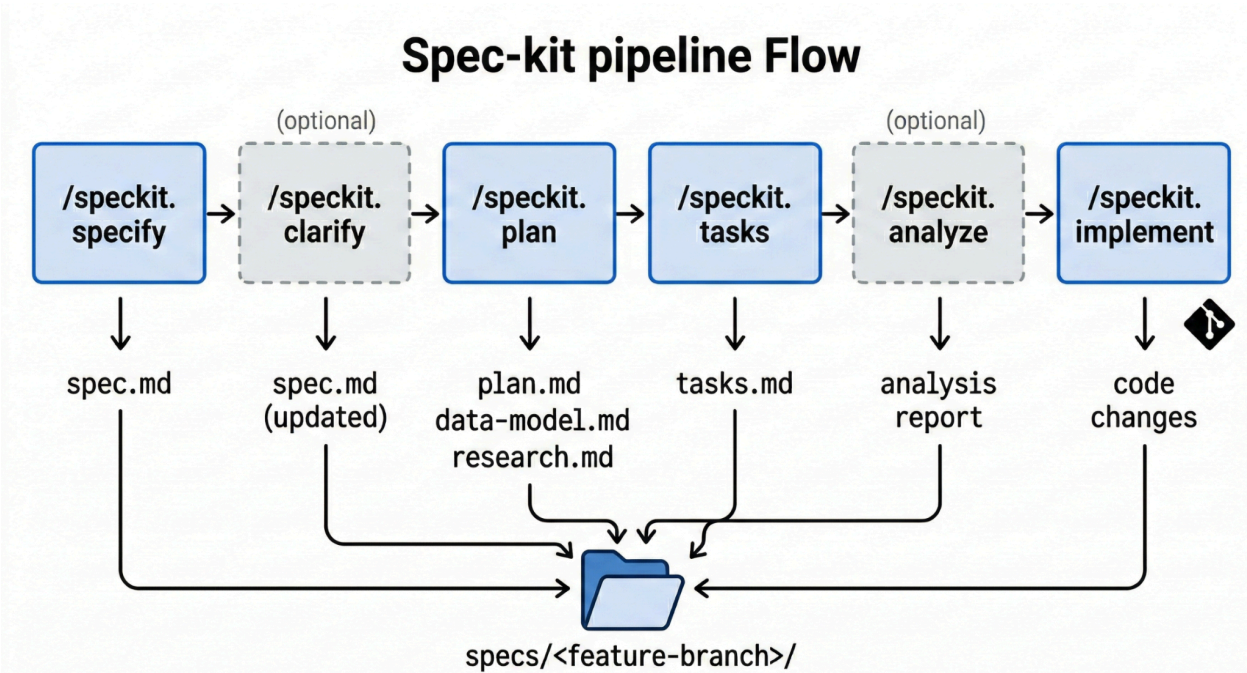


Antigravity 會要求您先審查並核准這些構件，再執行作業。這個「規劃然後實作」的迴圈是規格導向開發的核心：規格會引導程式碼，反之則否。

本程式碼研究室會以這個基礎為起點，進一步介紹以 [spec-kit](#) 為基礎的意見主導式版本控管工作流程。spec-kit 是 GitHub 提供的規格導向開發架構。每個功能都會經過審慎的管道，每個構件都是獨立的文件，您可以在 `git` 中查看、編輯及追蹤。管道包含兩個選用的品質閘階段 (釐清和分析)，可在問題造成實作問題前及早發現：

階段	構件	Purpose
<code>/speckit.specify</code>	<code>spec.md</code>	定義要建構的內容 (使用者導向，不限技術)
<code>/speckit.clarify</code> (選填)	更新時間： <code>spec.md</code>	找出規格不足的區域、提出具體的問題釐清疑慮，然後將答案編碼回規格
<code>/speckit.plan</code>	<code>plan.md</code> 、 <code>data-model.md</code> 、 <code>research.md</code>	設計建構方式 (技術方法、資料模型、研究)
<code>/speckit.tasks</code>	<code>tasks.md</code>	將計畫分解為可執行的步驟
<code>/speckit.analyze</code> (選填)	分析報告	在實作前，先檢查工作是否有風險、缺口或遺漏的極端情況

<code>/speckit.implement</code>	程式碼變更	執行各項工作，並逐一勾選
---------------------------------	-------	--------------



每個構件都會以檔案形式保留在 `specs/<feature-branch>/` 中，並在 Git 中進行版本控制，可重複使用。如果對話中斷，或是您想稍後再查看決策，規格文件隨時都能派上用場，不會埋沒在對話記錄中。

附註：Antigravity 內建構件在 UI 方面具有優勢，可讓使用者對生成的構件（甚至是圖片！）發表評論。）。因此在本教學課程中，我們會修改原始的 `spec-kit` 提示，以便與 `implement` 階段的內建構件整合

入門版存放區包含 `.agents/workflows/` 中的這些 SDD 工作流程，以及 `.specify/templates/` 中的範本。您稍後會使用這些值，為代理程式新增功能。

4. 使用 Antigravity 啟動專案環境

現在，讓我們模擬「更接近」日常工作的條件：

- 存放區管理不善
- README 檔案已過時
- 文件更新頻率不高

注意：如果這項資訊與你的情況不符，表示你非常幸運

在這種情況下，我們通常會先建立專案的地圖或情境，讓 Antigravity 著手處理。這個步驟提供一個範例，說明如何建立技能，分析存放區並產生專案環境文件，讓 Antigravity 深入瞭解現有程式碼集。

此外，這項工具也會設定專案章程，也就是 SDD 工作流程驗證時所依據的原則，這些原則不得協商。這些資訊會提供 Antigravity 後續 SDD 週期所需的背景資訊和限制

Antigravity 內容階層

Antigravity 使用三種層級的環境，各有不同的範圍：

- **規則** (`.agents/rules/`)：一律啟用的指令。工作區中的每項對話都會顯示這些規則 (如果已啟用)。使用規則提供專案的整體脈絡，例如架構決策、編碼標準或技術堆疊資訊。
- **技能** (`.agents/skills/`)：隨選知識。只有在目前工作符合技能的 `description` 欄位時，Antigravity 才會載入技能。使用技能取得特定領域的參考資源。
- **工作流程** (`.agents/workflows/`)：使用 `/` 指令觸發的已儲存提示。工作流程適用於可重複執行的多步驟程序，例如 SDD 管道。

注意：專案目前有工作流程 (SDD 管道) 和 2 項技能 (**repo-research** 和 **mcp-toolbox-postgres**)，但沒有規則。稍後，我們將建立 `project-context.md` 規則，讓 Antigravity 持續瞭解程式碼集，包括使用的技術、檔案的整理方式，以及目前已建構的內容。

啟用技能

範例存放區的 `.agents/skills/` 中包含兩項預先編寫的技能。這些檔案包含詳細說明，但開頭是 `TODO(code1ab)` 註解，而非必要的 YAML 前置事項。如果沒有前言，Antigravity 就無法探索這些內容。

反重力技能需要在檔案頂端提供 YAML 前言區塊，並包含兩個欄位：

- **name** - 技能的專屬 ID
- **description**：自然語言摘要，Antigravity 會比對這項摘要，決定要為特定要求載入哪個技能

在編輯器中開啟 `.agents/skills/mcp-toolbox-postgres/SKILL.md`。將頂端的 `TODO(code1ab)` 註解行替換為下列前言：

```
---
name: mcp-toolbox-postgres
description: Configure MCP Toolbox for PostgreSQL – sources, tools, and embedding models
---
```

注意：MCP Toolbox Postgres 技能包含 `sources`、`tools` 和 `embedding models` 的正確設定

在編輯器中開啟 `.agents/skills/repo-research/SKILL.md`。將頂端的 `TODO(codelab)` 註解行替換為下列前言：

```
---
name: repo-research
description: Analyze a repository's structure, technologies, and patterns to create or update
a project context document. Use when asked to research, analyze, or understand a codebase.
---
```

注意：請詳閱 `repo-research SKILL.md`。以下是分析存放區的提示範例，請注意，增量更新規則刻意採用保守做法。如果 Antigravity 在研究期間用盡情境，或對話中斷，目前為止發現的所有內容都會儲存到檔案中。重新叫用技能時，系統會從上次中斷的地方繼續，不會遺失進度。

警告！ 分析大型存放區可能會快速用盡配額！

確認兩個技能都有有效的前言：

```
head -4 .agents/skills/mcp-toolbox-postgres/SKILL.md
head -4 .agents/skills/repo-research/SKILL.md
```

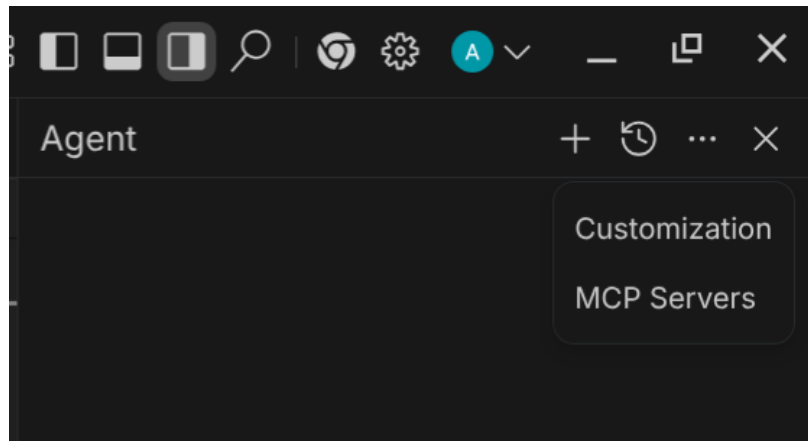
每個欄位都應顯示以 `---` 分隔符號括住的 `name:` 和 `description:` 欄位。如果缺少分隔符或欄位，Antigravity 就無法辨識技能。

這兩項技能都是視需求載入，Antigravity 會根據 `description` 欄位比對要求，並只在相關時提取完整指令。

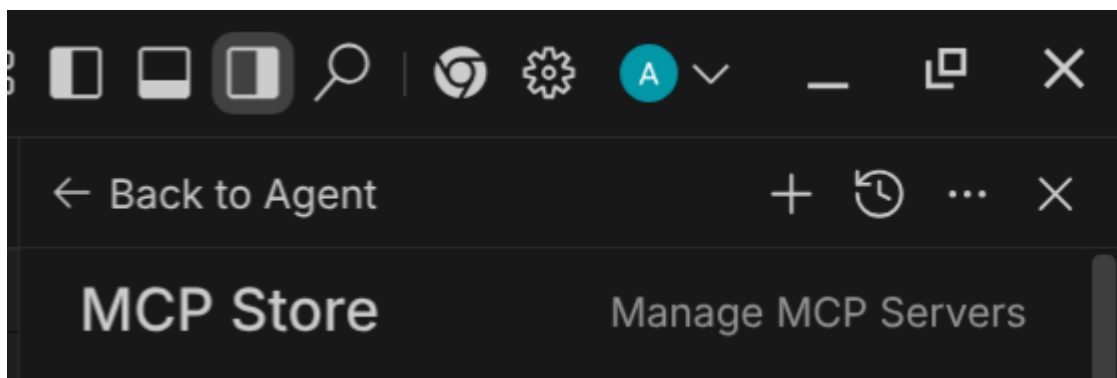
設定 ADK Docs MCP

為確保程式碼代理程式具備開發 ADK 代理程式的正確知識，必須授予適當的 ADK 文件存取權。我們可以設定 [ADK 文件 MCP 伺服器](#) 來啟用這項功能。

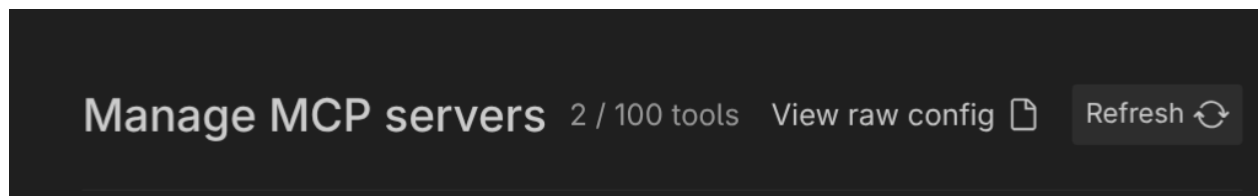
首先，在 Antigravity 編輯器中，找到代理程式視窗右上方的「省略號」圖示並點選，您會看到 MCP 的設定選項和自訂項目，如下所示。按一下 MCP。



接著，按一下 `Manage MCP servers` 按鈕



系統會開啟 MCP 伺服器設定分頁，然後按一下 `View Raw Config` 按鈕



系統會開啟 `mcp_config.json` 檔案。現在請將這項設定複製並貼到檔案中

```
{
  "mcpServers": {
    "adk-docs-mcp": {
      "command": "uvx",
      "args": [
        "--from",
        "mcpdoc",
        "mcpdoc",
        "--urls",
        "AgentDevelopmentKit:https://adk.dev/llms.txt",
        "--transport",
        "stdio"
      ]
    }
  }
}
```

注意：別忘了儲存變更！

完成這項設定後，程式碼編寫代理就能使用 MCP 伺服器搜尋及讀取 ADK 說明文件

安裝公開技能

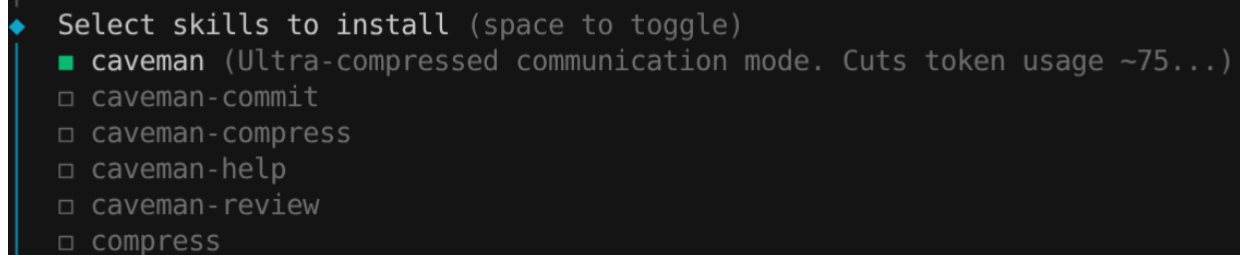
我們先前檢查了自行建立的技能 (`repo-research` 和 `mcp-toolbox-postgres` 技能)。不過，還有許多其他技能有助於支援軟體開發，而且可以輕鬆安裝至程式設計代理。

原始人技能

其中一項有助於減少權杖用量的實用技能是[原始人技能](#)。這項技能可以從代理程式中剪下輸出權杖，因此當您像本教學課程一樣使用完整的規格導向開發時，這項技能會是不錯的組合

```
npx skills add JuliusBrussee/caveman
```

與先前類似，我們現在先選取 `caveman` 技能



```
◆ Select skills to install (space to toggle)
■ caveman (Ultra-compressed communication mode. Cuts token usage ~75...)
□ caveman-commit
□ caveman-compress
□ caveman-help
□ caveman-review
□ compress
```

然後按一下 **Enter** 鍵以啟用預設安裝選項，並在目前專案中安裝技能

現在我們應該已設定好 4 項技能，可以開始 SDD 歷程

```
.agents/skills/
├── caveman
├── mcp-toolbox-postgres
└── repo-research
```

生成專案背景資訊

確認規則目錄是否存在：

```
mkdir -p .agents/rules
```

在 Antigravity 的代理程式管理員/聊天方塊 (在編輯器模式中按下 `ctrl + L`)，發起新的對話。Type:

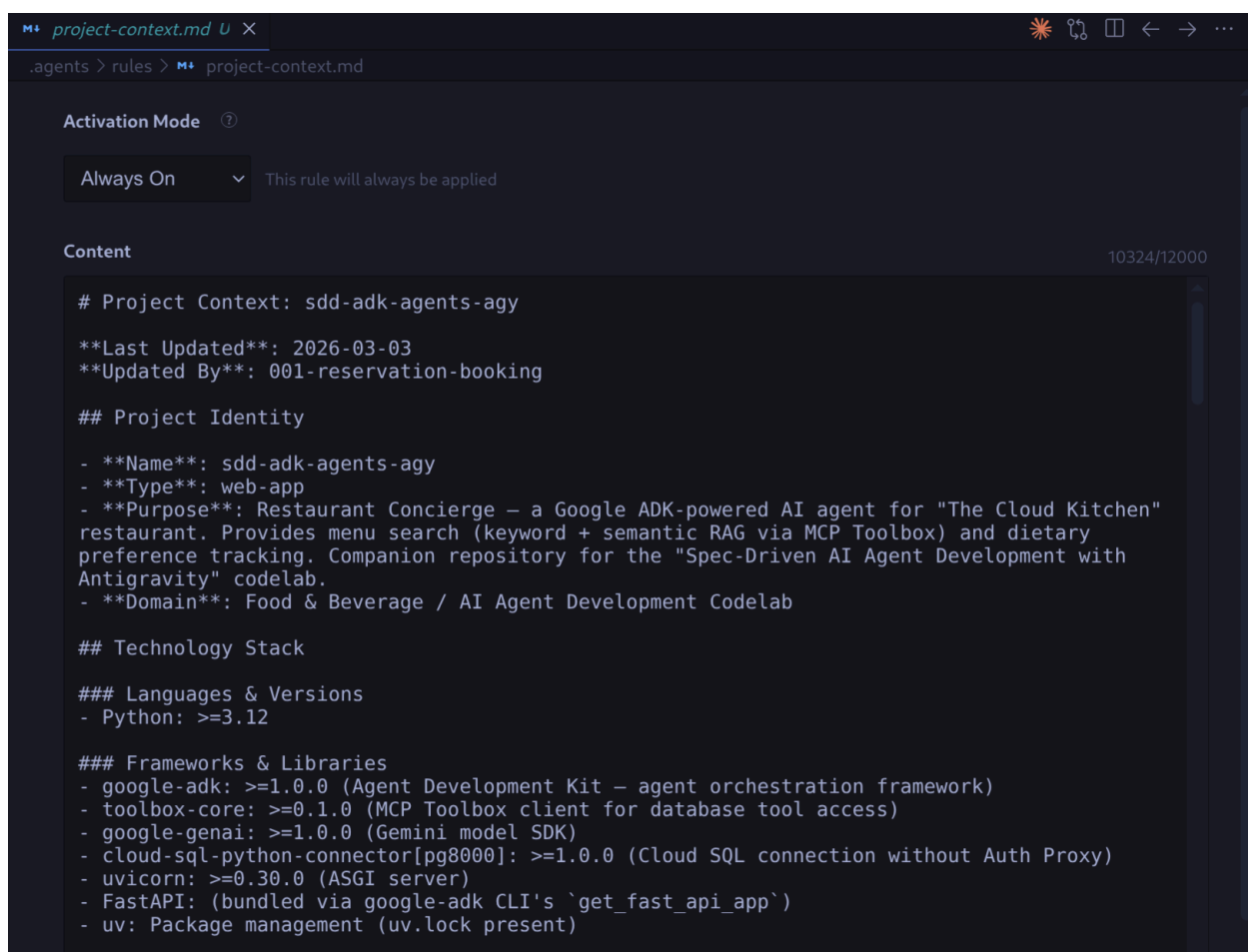
```
Research this repository and create a project context document, and communicate efficiently
```

Antigravity 會將您的要求與 `repo-research` 和 `caveman` 技能 (有效溝通的指令) 相符，然後開始有系統地分析程式碼集。這項工具會讀取設定檔、原始碼和說明文件，然後根據發現的內容填入專案環境範本。

完成後，在編輯器中開啟 `.agents/rules/project-context.md`。其中包含專案的具體資訊：技術堆疊 (Python 3.12、ADK、MCP Toolbox、Cloud SQL)、專案結構、資料模型 (含 `pgvector` 的 `menu_items` 表格) 和外部整合。

提示：專案背景文件會隨著專案進展而擴充。你可以重新執行技能，使用新加入的內容更新脈絡。增量更新規則可確保現有內容不會遭到覆寫，而是保留下來，並附加新技術、實體和整合項目。

如要進行特定更新，請使用特定提示：「研究資料庫結構定義並更新專案環境」或「使用新工具定義更新專案環境」。每次叫用都會在現有文件中新增內容。



```
# Project Context: sdd-adk-agents-agy

**Last Updated**: 2026-03-03
**Updated By**: 001-reservation-booking

## Project Identity

- **Name**: sdd-adk-agents-agy
- **Type**: web-app
- **Purpose**: Restaurant Concierge – a Google ADK-powered AI agent for "The Cloud Kitchen" restaurant. Provides menu search (keyword + semantic RAG via MCP Toolbox) and dietary preference tracking. Companion repository for the "Spec-Driven AI Agent Development with Antigravity" codelab.
- **Domain**: Food & Beverage / AI Agent Development Codelab

## Technology Stack

### Languages & Versions
- Python: >=3.12

### Frameworks & Libraries
- google-adk: >=1.0.0 (Agent Development Kit – agent orchestration framework)
- toolbox-core: >=0.1.0 (MCP Toolbox client for database tool access)
- google-genai: >=1.0.0 (Gemini model SDK)
- cloud-sql-python-connector[pg8000]: >=1.0.0 (Cloud SQL connection without Auth Proxy)
- uvicorn: >=0.30.0 (ASGI server)
- FastAPI: (bundled via google-adk CLI's `get_fast_api_app`)
- uv: Package management (uv.lock present)
```

設定專案章程

在規劃和分析期間，SDD 工作流程會參照 `.specify/memory/constitution.md` 的專案章程。`/speckit.plan` 工作流程會對其執行「憲法檢查」，並將違規事項標示為「重大」`/speckit.analyze`。如果將憲章留為空白範本並包含預留位置權杖，這些檢查就沒有可驗證的內容，因此計畫和分析會在沒有防護措施的情況下執行。

憲章定義了不可協商的專案原則。這是由單一開發人員維護的小型存放區，因此章程應反映該範圍，保持簡單一致，避免過度設計。

在 Antigravity 的 Agent Manager 中，發起新對話。執行憲法工作流程：

```
/speckit.constitution This is a small restaurant concierge ADK agent maintained by one
developer. Set 3 principles: (1) All database operations go through MCP Toolbox tool
definitions in tools.yaml – no raw SQL in Python code, no ORM. (2) Session state uses ADK
ToolContext – no custom state management, no external state stores. (3) Keep it simple –
follow existing file and naming conventions exactly.
```


Antigravity 會以具體原則填入憲法範本、指派版本 (1.0.0)，並對 SDD 範本執行一致性檢查。

在 `.specify/memory/constitution.md` 查看生成的章程。確認這三項原則是否明確列出。

```
constitution.md M X
.specify > memory > constitution.md > # sdd-adk-agents-agy Constitution
1  # sdd-adk-agents-agy Constitution
20
21  ## Core Principles
22
23  ### I. Tool-Driven Database Operations
24  All database interaction MUST occur through MCP Toolbox tool definitions in `tools.
25  yaml`. Writing raw SQL strings within Python code or using Object-Relational Mapping
26  (ORM) libraries is strictly forbidden. This ensures a clean separation between data
27  access logic and agent orchestration.
28
29  ### II. Native ADK State Management
30  Session state MUST be managed exclusively using the ADK `ToolContext`. Custom state
31  management solutions, external state stores (other than those provided by ADK), or
32  global variables for session tracking are not permitted. This maintains consistency
33  with the ADK framework's lifecycle and persistence model.
34
35  ### III. Radical Simplicity & Convention
36  Follow existing file structures, naming conventions, and project patterns exactly. Do
37  not introduce new architectural layers or abstractions unless absolutely necessary.
38  This project is maintained by a single developer; ease of maintenance through
39  familiarity and predictability is paramount.
40
41  ## Development Workflow
42
43  - **Specification First**: Every feature must be defined in a `spec.md` before
44  implementation.
45  - **Plan Review**: Implementation plans must be reviewed against these principles.
46  - **Tool-First Design**: New data requirements must result in new `tools.yaml` entries
47  before Python changes.
```

注意：我們刻意將憲法限制在 3 項原則，適用於獨立開發人員。 `/speckit.constitution` 工作流程支援增量修訂，只要使用其他原則再次運作執行，系統就會自動處理版本管理。如果是團隊專案，您會加入程式碼審查、測試紀律、觀測能力和 API 版本管理的原則。

檢查點： `.agents/rules/project-context.md` 存在，且包含存放區的具體資訊。 `.agents/skills/repo-research/SKILL.md` 說明如何使用 YAML 前言、指令和衝突解決方式建立技能。 `.specify/memory/constitution.md` 包含三項專案原則，計畫和分析工作流程會根據這些原則進行驗證。瞭解三層脈絡階層架構：規則 (一律啟用)、技能 (隨選) 和工作流程 (斜線指令)。

5. SDD 週期 - 新增預留功能

這個步驟會逐步說明完整的 SDD 週期，將預訂訂位功能新增至餐廳服務專員。您會帶領 Antigravity 經歷每個階段 (指定、釐清、規劃、工作、分析、實作)，觀察每個構件如何以前一個構件為基礎。這是本程式碼研究室的核心學習體驗。

重要事項！ Antigravity 是 AI 助理，因此生成的構件 (規格、計畫、工作、程式碼) 可能與本教學課程所述不同。檔案名稱、確切的 SQL 結構定義和程式碼結構可能有所不同。這是正常情況，也是 AI 輔助開發的一部分。SDD 工作流程可確保您在每個構件成為程式碼前都經過審查，因此請自行判斷是否接受、修改或重新生成任何不符合預期的輸出內容。

指定功能

在 Antigravity 的 Agent Manager 中，發起新對話。輸入 `/speckit.specify` 工作流程指令，並提供功能說明：

```
/speckit.specify Add reservation booking capability to the restaurant concierge agent. Guests should be able to make a table reservation by providing their name, party size, date, and time. They should also be able to check existing reservations. The agent should confirm reservation details before booking and handle special requests (e.g., "window seat", "birthday celebration").
```

Antigravity 會建立功能分支、產生規格文件，並執行品質驗證。如果 Antigravity 提出需要釐清的問題，請根據上述功能說明回答。

注意：完成規格後，Antigravity 可能會提供「制定技術計畫」或「釐清規格需求」等交接事項。這些是下一個工作流程階段的提示 (`/speckit.plan` 或 `/speckit.clarify`)

這項規格的重點在於「內容」和「原因」，而非「方式」。這段文字說明瞭使用者體驗 (「訪客可以提供姓名、人數、日期和時間來預訂座位」)，但未提及 SQL 資料表、`tools.yaml` 或 ADK API。實作詳細資料會在規劃階段提供。

在 `specs/<branch-name>/spec.md` 查看生成的規格。確認是否擷取功能需求和成功條件。


```
spec.md 0 X
specs > 001-reservation-booking > spec.md > # Feature Specification: Reservation Booking
1 # Feature Specification: Reservation Booking
2
3 **Feature Branch**: `001-reservation-booking`
4 **Created**: 2026-03-03
5 **Status**: Draft
6 **Input**: User description: "Add reservation booking capability to the restaurant
  concierge agent. Guests should be able to make a table reservation by providing their
  name, party size, date, and time. They should also be able to check existing
  reservations. The agent should confirm reservation details before booking and handle
  special requests (e.g., \"window seat\", \"birthday celebration\")."
7
8 ## Clarifications
9
10 ### Session 2026-03-03
11 - Q: What are the minimum and maximum party sizes allowed for a standard reservation? →
  A: 1 to 12.
12 - Q: How far in advance can a guest book a table, and is there a cut-off time for
  same-day bookings? → A: Up to 3 months in advance; no same-day bookings after 4 PM.
13 - Q: How should the system handle a request that matches an existing reservation? → A:
  Block duplicates and inform the guest.
14
15 ## User Scenarios & Testing *(mandatory)*
16
17 ### User Story 1 - Book a New Table Reservation (Priority: P1)
18
19 As a guest, I want to book a table for a specific date and time so that I can ensure I
  have a seat at the restaurant.
20
21 **Why this priority**: Core value of the feature; primary reason for the reservation
  system.
```

釐清規格 (選填)

執行釐清工作流程，找出並解決規格中未明確指定的部分：

```
/speckit.clarify
```

Antigravity 會掃描規格，找出模糊不清、缺少驗收條件和規格不足之處。並提出具體的釐清問題，每個問題都可用簡短的選項或片語回答。您的答案會直接編碼回規格，因此在開始規劃前，規格會更加精確。

提示：釐清步驟為選用步驟，但建議用於複雜功能。如果規格很簡單，可以直接跳到規劃步驟。

規劃實作作業

執行規劃工作流程：

```
/speckit.plan Update the restaurant concierge agent to multi agent architecture which
separate responsibilities for handling menu and reservations. Utilize your access to ADK Docs
tools and MCP toolbox skill to implement it properly
```

Antigravity 會透過兩個階段生成技術計畫：

1. 研究階段：解決現有程式碼集中的未知問題，產生 `research.md`
2. 設計階段 - 建立 `data-model.md` (預訂實體定義) 並更新 `project-context.md`

Antigravity 應在規劃期間使用 ADK 文件 MCP 工具和 MCP 工具箱文件技能。查看重要構件：

- `specs/<branch-name>/plan.md` - 技術方法：要修改哪些檔案、要遵循哪些模式
- `specs/<branch-name>/data-model.md` - 預訂實體定義 (資料欄、類型、關係)
- `specs/<branch-name>/research.md` - 決策和理由

```

plan.md U X
specs > 001-reservation-booking > plan.md > # Implementation Plan: Reservation Booking
1 # Implementation Plan: Reservation Booking
2
3 **Branch**: `001-reservation-booking` | **Date**: 2026-03-03 | **Spec**: [spec.md]
  (file:///home/alvinprayuda/Work/sdd-adk-agents-agy/specs/001-reservation-booking/spec.
  md)
4 **Input**: Feature specification from `/specs/001-reservation-booking/spec.md`
5
6 ## Summary
7 Implement a table reservation system allowing guests to book tables (1-12 people,
  3-month window) and check existing bookings by name. Includes duplicate prevention and
  special request handling.
8
9 ## Technical Context
10
11 **Language/Version**: Python >=3.12
12 **Primary Dependencies**: google-adk, toolbox-core, google-genai,
  cloud-sql-python-connector
13 **Storage**: Cloud SQL (PostgreSQL) - New `reservations` table
14 **Testing**: Manual via Chat UI + SQL verification
15 **Project Type**: web-app (ADK Agent)
16 **Performance Goals**: N/A (Internal/Codelab tool)
17 **Constraints**: Tool-driven DB access, Native ADK state
18 **Scale/Scope**: Single restaurant concierge
19
20 ## Constitution Check
21
22 *GATE: Must pass before Phase 0 research. Re-check after Phase 1 design.*
23
24 - [x] **Tool-Driven DB**: Are all database operations defined in `tools.yaml`? (No raw
  SQL/ORM) - Checked: `create_reservation`, `get_reservations_by_name`,
  `check_duplicate_reservation` defined in `tools.yaml`.

```

生成工作

執行工作流程

```
/speckit.tasks
```

Antigravity 會將計畫分成 `specs/<branch-name>/tasks.md` 中的排序工作清單。工作會採用嚴格的檢查清單格式，包含 ID、優先順序標記和檔案路徑，例如：

- ```

- [] [T001] [P] Create reservations table schema in scripts/seed_db.py
- [] [T002] [P] Add create_reservation tool to tools.yaml
- [] [T003] [P] Add list_reservations tool to tools.yaml
- [] [T004] [P] Update agent instruction in restaurant_concierge/agent.py

```

工作會依階段分類：設定 → 基礎 → 使用者故事 → 潤飾。掃描工作清單，瞭解要建立和修改的項目。

```
tasks.md U X
specs > 001-reservation-booking > tasks.md > # Tasks: Reservation Booking
1 # Tasks: Reservation Booking
2
3 **Input**: Design documents from `/specs/001-reservation-booking/`
4 **Prerequisites**: plan.md (required), spec.md (required), research.md, data-model.md,
 contracts/
5
6 ## Phase 1: Setup (Shared Infrastructure)
7
8 **Purpose**: Initial project alignment and context propagation.
9
10 - [] T001 Verify project constitution compliance in `specs/001-reservation-booking/
 plan.md`
11 - [] T002 [P] Sync `project-context.md` with recent feature history and new entities
12
13 ---
14
15 ## Phase 2: Foundational (Blocking Prerequisites)
16
17 **Purpose**: Core database and tool infrastructure.
18
19 **⚠ CRITICAL**: Database schema must be present before agent logic can interact with
 it.
20
21 - [] T003 [P] Update `scripts/seed_db.py` to include `reservations` table schema (id,
 guest_name, party_size, reservation_datetime, special_requests, created_at)
22 - [] T004 Run schema update command: `uv run python scripts/seed_db.py`
23 - [] T005 [P] Update `tools.yaml` with SQL tools: `create_reservation`,
 `get_reservations_by_name`, and `check_duplicate_reservation`
24 - [] T006 Verify Toolbox loads tools: `toolbox --tools-file tools.yaml` (check for no
 syntax errors)
```

## 分析工作 (選用)

執行分析工作流程，檢查工作是否有風險和缺口：

```
/speckit.analyze
```

Antigravity 會根據規格和計畫檢查工作清單，找出缺少的極端情況、可能衝突的工作，或是規格需求與計畫工作之間的落差。請先解決重大問題，再進行實作。

**提示：**如果工作清單很長，或功能涉及多個子系統，分析步驟就非常實用。如果是預訂等特定功能，快速掃描就足夠。

步驟 6 · 約 0 分鐘

## 6. 實作

執行導入工作流程：

```
/speckit.implement
```

Antigravity 會提供最終實作計畫和工作構件。請審查並核准，以繼續操作

提示：你可以直接留言！這項操作只能透過 Antigravity 內建的構件執行

Task

Task 28 minutes ago

Review

### Implementation Tasks: Reservation Booking

#### Phase 1: Setup

- ☐ T001 Verify constitution compliance
- ☐ T002 [P] Sync project-context.md

#### Phase 2: Foundational

- ☐ T003 [P] Update `scripts/seed_db.py` (reservations table)
- ☐ T004 Run schema update: `uv run python scripts/seed_db.py`
- ☐ T005 [P] Update `tools.yaml` (3 new SQL tools)
- ☐ T006 Verify Toolbox loads: `toolbox --tools-file tools.yaml`

#### Phase 3: US1 — Book Reservation (P1 MVP)

- ☐ T007 [P] [US1] Update system instruction in `agent.py`
- ☐ T008 [US1] Implement `book_reservation` tool in `agent.py`
- ☐ T009 [US1] Register tool in `root_agent`
- ☐ T010 [US1] Verify booking via Chat UI

#### Phase 4: US2 — Check Reservations (P2)

- ☐ T011 [P] [US2] Update instruction to use `get_reservations_by_name`
- ☐ T012 [US2] Verify lookup via Chat UI

Implementation Plan

Implementation Plan 29 minutes ago

Review

Proceed

## Implementation: Reservation Booking

Branch: 001-reservation-booking | Tasks: /specs/001-reservation-booking/tasks.md

### Checklist Status

| Checklist       | Total | Completed | Status |
|-----------------|-------|-----------|--------|
| requirements.md | 19    | 19        | ✓ PASS |

### Phase 2: Foundational Changes

T003 — scripts/seed\_db.py

Add reservations table creation inside create\_schema() :

```
python

cursor.execute("""
 CREATE TABLE IF NOT EXISTS reservations (
 id SERIAL PRIMARY KEY,
 guest_name VARCHAR(255) NOT NULL,
 party_size INTEGER NOT NULL,
 reservation_datetime TIMESTAMP NOT NULL,
 special_requests TEXT,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 UNIQUE (guest_name, reservation_datetime)
);
""")
```

Antigravity 會執行任務，並在完成後逐一勾選。完成後，系統會顯示完整的導覽

Implementation Plan Walkthrough X tasks.md U agent.py M

Walkthrough less than a minute ago Review

### Walkthrough: Reservation Booking

Branch: 001-reservation-booking | Date: 2026-03-03

#### What Was Done

All code changes for the Reservation Booking feature are complete. Three files were modified:

- scripts/seed\_db.py — New Table Schema  
Added a `reservations` table inside `create_schema()` with a `UNIQUE (guest_name, reservation_datetime)` constraint to enforce the duplicate-blocking requirement (FR-010).
- tools.yaml — Three New SQL Tools

| Tool                                     | Operation          | Purpose                                             |
|------------------------------------------|--------------------|-----------------------------------------------------|
| <code>create_reservation</code>          | INSERT + RETURNING | Creates a new booking record                        |
| <code>get_reservations_by_name</code>    | SELECT ILIKE       | Retrieves bookings by guest name (case-insensitive) |
| <code>check_duplicate_reservation</code> | SELECT COUNT       | Checks for duplicate before booking                 |

- restaurant\_concierge/agent.py — `book_reservation` Tool + Updated Instructions
  - Added `book_reservation` async tool that: validates party size (1-12), checks for duplicates, and calls `create_reservation` via Toolbox.

## 測試程式碼變更

導入完成後，請確認已進行重要變更。實際的檔案名稱和內容可能有所不同，但這些模式應會出現在 `tools.yaml` 和 `agent.py` 中：

```
Verify reservation tools were added to tools.yaml
grep -i "reservation" tools.yaml
```

輸出內容會類似這樣

```
...
get_reservations_by_name:
 Retrieve all reservations for a guest by their name. Uses case-insensitive
 SELECT id, guest_name, party_size, reservation_datetime, special_requests,
 created_at
 FROM reservations
 ORDER BY reservation_datetime DESC
...
```

如果是 `agent.py`

```
Verify agent instruction was updated
grep -i "reservation" restaurant_concierge/agent.py

Check what files changed
git diff --name-only
```

您可能會發現類似的變更

```
...
- Table Reservations: Help guests book a table or check their existing
reservations.
Reservation Booking Rules
When a guest wants to make a reservation, collect ALL of the following before
confirming:
IMPORTANT: Before calling `book_reservation`, you MUST:
- Only call `book_reservation` after the guest says "yes" or "confirm"
Checking Reservations
When a guest asks to check their reservations:
- Use `get_reservations_by_name` to find their bookings
 book_reservation,
...
```

**注意：**如果 `grep` 未傳回任何檔案的相符項目，實作可能使用了不同的命名方式。檢查 `git diff`，查看完整變更內容，並確認是否已新增預訂相關邏輯。

這些變更應會影響種子資料庫指令碼。更新後的指令碼應會建立 `reservations` 資料表 (如果該資料表尚不存在)。您應該會看到輸出內容，確認已建立新資料表 (現有 `menu_items` 資料會保留)。

**注意：**Antigravity 可能會建立獨立的遷移指令碼 (例如 `scripts/create_reservations.py`)，而不是更新 `seed_db.py`。檢查 `git diff --name-only` 中是否有任何新指令碼，並改為執行這些指令碼。 `scripts/`

如果到目前為止一切順利，我們可以在 ADK 代理程式開發人員 UI 中測試這項功能。讓我們進行資料庫遷移，納入新資料表，並重新啟動工具箱，以便在 `tools.yaml` 中擷取新工具定義。停止所有現有的 Toolbox 程序，然後重新啟動：

```
lsof -ti:5000 | xargs kill -9 2>/dev/null; echo "Done"
./scripts/setup_database.sh > database_setup.log 2>&1 &
```

您可以在 `database_setup.log` 中查看輸出記錄，確認 Antigravity 是否產生正確的程式碼和設定。如果發生錯誤，請嘗試與代理程式互動來修正問題

如果一切順利，我們就可以啟動 ADK 開發 UI：

```
uv run adk web .
```

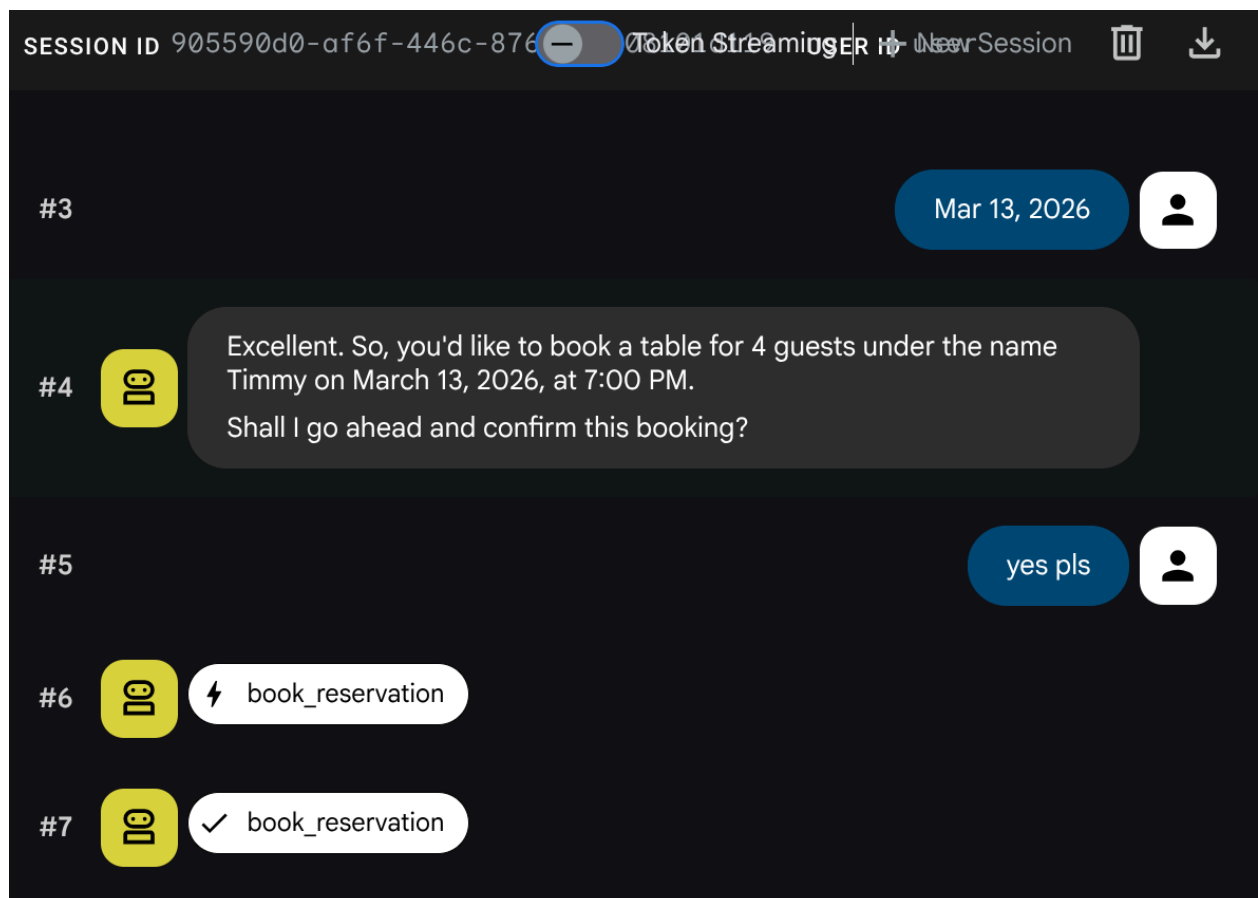
在瀏覽器中開啟 `http://localhost:8000`，並使用下列提示進行測試：

```
I'd like to book a table for 4 people on Friday at 7pm under the name Timmy
```

**檢查：**服務專員應確認預約詳細資料並建立預約

```
Do I have any upcoming reservations?
```

**檢查：**代理程式應會列出您剛建立的預訂





#1

Do I have any upcoming reservations?



#2



Certainly! To check for your upcoming reservations, could you please tell me the name under which the reservation was made?

#3

Timmy



#4



⚡ get\_reservations\_by\_name

#5



✓ get\_reservations\_by\_name

#6

continue



#7



Yes, Timmy, you have two upcoming reservations:

- **March 7, 2025, at 7:00 PM** for a party of 4.
- **March 13, 2026, at 7:00 PM** for a party of 4.

Is there anything else I can assist you with regarding these reservations or anything else?

現在，請按兩次 `Ctrl+C` 停止 ADK 開發人員使用者介面。

**查核點：**代理程式會處理預訂和查詢作業。您已完成完整的 SDD 週期：指定 → 釐清 → 規劃 → 任務 → 分析 → 實作 → 測試。每個階段都會在 `specs/<branch-name>/` 中產生版本化構件。

---

## 7. 挑戰 (選填)

您現在已瞭解完整的 SDD 工作流程。測試：

- 執行第二個 SDD 週期，為餐廳服務人員建構網路即時通訊介面，這次不提供逐步指引。
- 將代理程式部署至 Cloud Run，用於實際工作環境

## 提示

- 專案沒有前端架構。Antigravity 應建議使用原始 HTML/CSS/JS，如果建議使用 React 或類似項目，請引導它朝簡單的方向發展 (您的憲章「保持簡單」原則應會發現這個問題)。
  - ADK 伺服器會公開 `/run_sse` (用於串流) 和 `/apps/{app_name}/users/{user_id}/sessions` (用於工作階段管理)。Antigravity 會從專案環境中探索這些項目。
  - 實作完成後，請使用 `uv run uvicorn server:app --host 0.0.0.0 --port 8080` (而非 `adk web`) 啟動伺服器，確保靜態檔案掛接作業正常運作。
  - 請前往 <http://localhost:8080/static/index.html> 進行測試。
  - 參考程式碼實驗室已說明如何部署及保存 ADK 代理程式，請將 Antigravity 參照這個程式碼實驗室！
-

## 8. 恭喜！

您已透過 Antigravity 的 SDD 工作流程，擴充餐廳服務專員 ADK 代理程式的預訂功能，完全不必手動編寫應用程式程式碼。

### 您建構的內容

- 餐廳服務專員 ADK 代理，可提供菜單搜尋、語意搜尋、飲食偏好追蹤和預訂服務
- Antigravity 技能，可進行存放區研究，並產生及維護專案內容文件
- 專案章程，在規劃和分析期間強制執行不可協商的原則
- 完整的 SDD 週期，展示指定 → 釐清 → 規劃 → 工作 → 分析 → 實作工作流程

### 您學到的內容

- 瞭解如何在 Antigravity 中使用 Spec-Driven Development 工作流程，有系統地將功能新增至現有程式碼集
- 如何建立 Antigravity 技能，以及設定 MCP，將網域知識封裝起來，以便在對話中重複使用
- 如何啟動專案環境，讓 Antigravity 針對架構、模式和技術選擇做出明智決策
- 如何設定 SDD 工作流程驗證的專案章程
- 如何透過 MCP Toolbox，使用新的資料庫支援工具擴充 ADK 代理

### 清除所用資源

停止所有正在執行的本機程序 (Toolbox)：

```
lsof -ti:5000 | xargs kill -9 2>/dev/null
```

刪除 Cloud SQL 執行個體，避免持續產生費用：

```
gcloud sql instances delete restaurant-db --quiet
```

(選用) 刪除整個專案：

```
gcloud projects delete $GOOGLE_CLOUD_PROJECT
```

**重要事項：**您可以使用這個指令 `gcloud projects undelete ${GOOGLE_CLOUD_PROJECT}`，在一段時間內復原這項刪除作業